Computer Graphics Practicals

Code, Output and Ubuntu Commands
Practical No. 2 to Practical No. 8

Note: Practical No. 1 was not shared, so this document covers Practicals 2-8. For Practicals 2, 3 and 4 the theory already matched the code, so no correction was needed. For Practicals 5, 6, 7 and 8 the original code is shown first, followed by the corrected code that matches the theory, each with its output.

Practical No. 2 - DDA and Bresenham Line Drawing Algorithms

Code

```
#include <graphics.h>
#include <iostream>
#include <cmath>
using namespace std;

void putStyledPixel(int x, int y, int style, int count) {
    if (style == 1) { // Simple
        putpixel(x, y, WHITE);
    }
    else if (style == 2) { // Dotted
        if (count % 2 == 0)
            putpixel(x, y, WHITE);
    }
    else if (style == 3) { // Dashed
        if ((count / 5) % 2 == 0) // draw 5 pixels, skip 5 pixels
            putpixel(x, y, WHITE);
    }
    else if (style == 4) { // Solid (same as simple)
        putpixel(x, y, WHITE);
    }
}

// ====================== DDA ======================
void DDA(int x0, int y0, int x1, int y1, int style) {
    int dx = x1 - x0;
    int dy = y1 - y0;
    int steps = max(abs(dx), abs(dy));
    float Xinc = dx / (float)steps;
    float Yinc = dy / (float)steps;

    float x = x0;
    float y = y0;
    for (int i = 0; i <= steps; i++) {
        putStyledPixel(round(x), round(y), style, i);
        x += Xinc;
        y += Yinc;
    }
}

// ================== Bresenham ====================
void Bresenham(int x0, int y0, int x1, int y1, int style) {
    int dx = abs(x1 - x0);
    int dy = abs(y1 - y0);
    int sx = (x0 < x1) ? 1 : -1;
    int sy = (y0 < y1) ? 1 : -1;
    int err = dx - dy;
    int count = 0;
    while (true) {
        putStyledPixel(x0, y0, style, count);
        if (x0 == x1 && y0 == y1) break;
        int e2 = 2 * err;
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;

        }
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
        count++;
    }
}

// ================== Main ======================
int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    cout << "======== Line Drawing Algorithms ========\n";
    cout << "1. DDA Algorithm\n2. Bresenham Algorithm\n";
    cout << "Enter your choice (1 or 2): ";
    int algo; cin >> algo;
    cout << "======== Line Styles ========\n";
    cout << "1. Simple Line\n2. Dotted Line\n3. Dashed Line\n4. Solid Line\n";
    cout << "Enter your choice (1-4): ";
    int style; cin >> style;
```

```
    int x0, y0, x1, y1;
    cout << "Enter starting point (x0, y0): ";
    cin >> x0 >> y0;
    cout << "Enter ending point (x1, y1): ";
    cin >> x1 >> y1;
    if (algo == 1)
        DDA(x0, y0, x1, y1, style);
    else
        Bresenham(x0, y0, x1, y1, style);
    getch();
    closegraph();
    return 0;
}
```

Output



DDA/Bresenham - simple, dotted, dashed and solid lines (line 50,50 to 250,150)

Theory already matches the code - no correction needed.

Practical No. 3 - Line Drawing in Four Quadrants

Code

```
#include <graphics.h>
#include <iostream>
using namespace std;

void screenToQuadrant(int sx, int sy, int &qx, int &qy, int xc, int yc) {
    qx = sx - xc;
    qy = yc - sy;
}

int main() {
    // Initialize graphics window
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    // Set background to black
    setbkcolor(BLACK);
    cleardevice();

    // Calculate center
    int screen_width = getmaxx();
    int screen_height = getmaxy();
    int xc = screen_width / 2;
    int yc = screen_height / 2;

    // Draw axes (in white)
    setcolor(WHITE);
    line(xc, 0, xc, screen_height);
    line(0, yc, screen_width, yc);

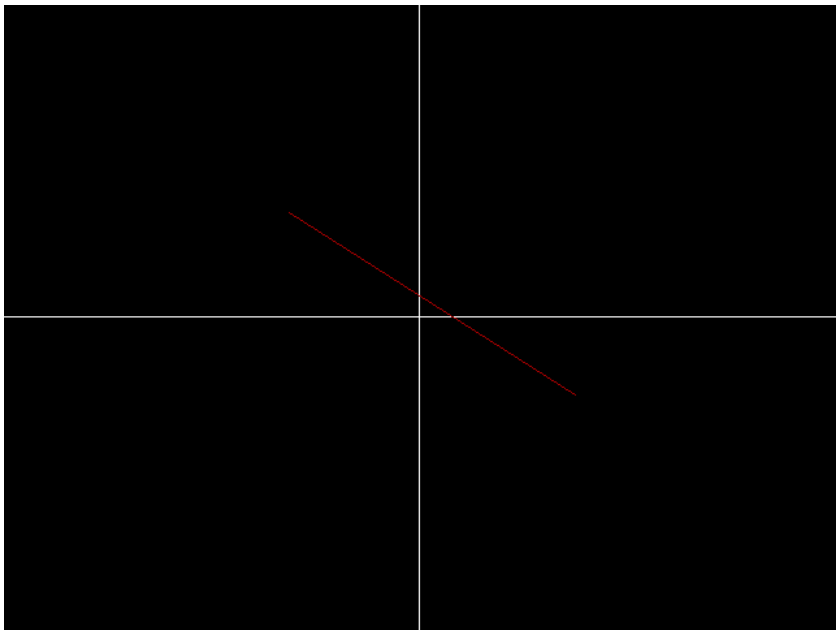
    // Input points in quadrant coordinates
    int qx1, qy1, qx2, qy2;

    cout << "Enter first point (x y) in quadrant coordinates: ";
    cin >> qx1 >> qy1;
    cout << "Enter second point (x y) in quadrant coordinates: ";
    cin >> qx2 >> qy2;

    // Convert to screen coordinates
    int sx1 = qx1 + xc;
    int sy1 = yc - qy1;
    int sx2 = qx2 + xc;
    int sy2 = yc - qy2;

    // Draw points and line (in red)
    setcolor(RED);
    putpixel(sx1, sy1, RED);
    putpixel(sx2, sy2, RED);
    line(sx1, sy1, sx2, sy2);
    cout << "Line drawn between (" << qx1 << "," << qy1 << ") and (" << qx2 << "," << qy2 << ")." << endl;
    getch();
    closegraph();
    return 0;
}
```

Output



Red line from quadrant II (-100,80) to quadrant IV (120,-60) with axes

Theory already matches the code - no correction needed.

Practical No. 4 - Bresenham's Circle Drawing Algorithm

Code

```
#include <graphics.h>
#include <iostream>
using namespace std;

void plotCirclePoints(int xc, int yc, int x, int y, int color) {
    putpixel(xc + x, yc + y, color);
    putpixel(xc - x, yc + y, color);
    putpixel(xc + x, yc - y, color);
    putpixel(xc - x, yc - y, color);
    putpixel(xc + y, yc + x, color);
    putpixel(xc - y, yc + x, color);
    putpixel(xc + y, yc - x, color);
    putpixel(xc - y, yc - x, color);
}

// Bresenham Circle Drawing
void drawBresenhamCircle(int xc, int yc, int r, int color) {
    int x = 0, y = r;
    int d = 3 - 2 * r;
    while (x <= y) {
        plotCirclePoints(xc, yc, x, y, color);
        if (d < 0) {
            d = d + 4 * x + 6;
        } else {
            d = d + 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}

void drawAxis(int xc, int yc, int xmax, int ymax) {
    setcolor(RED);
    line(0, yc, xmax, yc); // X-axis
    line(xc, 0, xc, ymax); // Y-axis
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    // Set black background
    setbkcolor(BLACK);
    cleardevice();

    int xmax = getmaxx();
    int ymax = getmaxy();
    int midx = xmax / 2;
    int midy = ymax / 2;

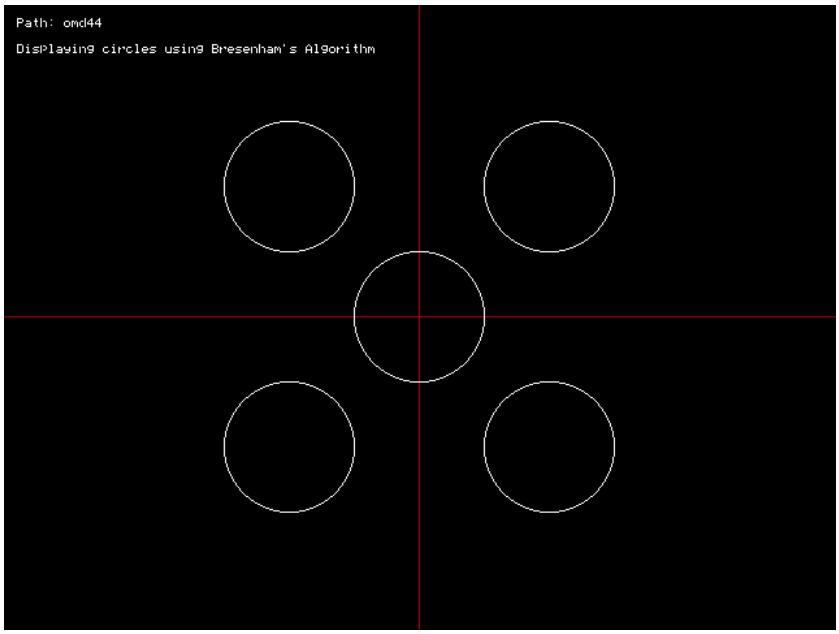
    // Draw axes
    drawAxis(midx, midy, xmax, ymax);

    // User input

    int r;
    cout << "Enter radius of object (circle): ";
    cin >> r;

    // Display path name
    setcolor(WHITE);
    outtextxy(10, 10, "Path: omd44");
    outtextxy(10, 30, "Displaying circles using Bresenham's Algorithm");
    int offset = 100;
    setcolor(WHITE);
    drawBresenhamCircle(midx, midy, r, WHITE); // Center
    drawBresenhamCircle(midx + offset, midy + offset, r, WHITE); // Q1
    drawBresenhamCircle(midx - offset, midy + offset, r, WHITE); // Q2
    drawBresenhamCircle(midx - offset, midy - offset, r, WHITE); // Q3
    drawBresenhamCircle(midx + offset, midy - offset, r, WHITE); // Q4
    getch();
    closegraph();
    return 0;
}
```

Output



Five circles (centre + one per quadrant), radius 50, red axes

Theory already matches the code - no correction needed.

Practical No. 5 - Clipping and Windowing Algorithms

Original Code - 1. Point Clipping

```
#include <graphics.h>
#include <iostream>
using namespace std;

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    int Xmin = 100, Ymin = 100, Xmax = 400, Ymax = 300;
    rectangle(Xmin, Ymin, Xmax, Ymax); // Clipping window

    int x, y;
    cout << "Enter point (x y): ";
    cin >> x >> y;
    if (x >= Xmin && x <= Xmax && y >= Ymin && y <= Ymax) {
        putpixel(x, y, GREEN); // Accepted
    } else {
        putpixel(x, y, RED); // Rejected
        cout << "Point rejected!\n";
    }
    getch();
    closegraph();
    return 0;
}
```

Original Code - 2. Cohen-Sutherland Line Clipping

```
#include <graphics.h>
#include <iostream>
using namespace std;

const int INSIDE = 0; // 0000
const int LEFT = 1;   // 0001
const int RIGHT = 2;  // 0010
const int BOTTOM = 4; // 0100
const int TOP = 8;    // 1000

int Xmin = 100, Ymin = 100, Xmax = 400, Ymax = 300;

int computeCode(int x, int y) {
    int code = INSIDE;
    if (x < Xmin) code |= LEFT;
    else if (x > Xmax) code |= RIGHT;
    if (y < Ymin) code |= BOTTOM;
    else if (y > Ymax) code |= TOP;
    return code;
}

void cohenSutherlandClip(int x1, int y1, int x2, int y2) {
    int code1 = computeCode(x1, y1);
    int code2 = computeCode(x2, y2);
    bool accept = false;

    while (true) {
        if ((code1 == 0) && (code2 == 0)) {
            accept = true;
            break;
        } else if (code1 & code2) {
            break;
        } else {
            int codeOut = code1 ? code1 : code2;

            int x, y;
            if (codeOut & TOP) {
                x = x1 + (x2 - x1) * (Ymax - y1) / (y2 - y1);
                y = Ymax;
            } else if (codeOut & BOTTOM) {
                x = x1 + (x2 - x1) * (Ymin - y1) / (y2 - y1);
                y = Ymin;
            } else if (codeOut & RIGHT) {
                y = y1 + (y2 - y1) * (Xmax - x1) / (x2 - x1);
                x = Xmax;
            } else {
                y = y1 + (y2 - y1) * (Xmin - x1) / (x2 - x1);
                x = Xmin;
            }
            if (codeOut == code1) {
                x1 = x; y1 = y; code1 = computeCode(x1, y1);
            } else {
                x2 = x; y2 = y; code2 = computeCode(x2, y2);
            }

        }
    }
    rectangle(Xmin, Ymin, Xmax, Ymax);
    if (accept) line(x1, y1, x2, y2);
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    int x1, y1, x2, y2;
    cout << "Enter line endpoints (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;

    line(x1, y1, x2, y2); // Original line
    delay(1000);
    cleardevice();
    cohenSutherlandClip(x1, y1, x2, y2);
    getch();
    closegraph();
    return 0;
}
```

Original Code - 3. Liang-Barsky Line Clipping

```
#include <graphics.h>
#include <iostream>
using namespace std;

int Xmin = 100, Ymin = 100, Xmax = 400, Ymax = 300;

bool clipTest(float p, float q, float &u1, float &u2) {
    float r;
    if (p == 0 && q < 0) return false;
    r = q / p;
    if (p < 0) {
        if (r > u2) return false;
        else if (r > u1) u1 = r;
    } else if (p > 0) {
        if (r < u1) return false;
        else if (r < u2) u2 = r;
    }
    return true;
}

void liangBarskyClip(int x1, int y1, int x2, int y2) {
    float dx = x2 - x1, dy = y2 - y1;
    float u1 = 0.0, u2 = 1.0;

    if (clipTest(-dx, x1 - Xmin, u1, u2))
    if (clipTest(dx, Xmax - x1, u1, u2))
    if (clipTest(-dy, y1 - Ymin, u1, u2))
    if (clipTest(dy, Ymax - y1, u1, u2)) {
        int nx1 = x1 + u1 * dx;
        int ny1 = y1 + u1 * dy;
        int nx2 = x1 + u2 * dx;
        int ny2 = y1 + u2 * dy;
        rectangle(Xmin, Ymin, Xmax, Ymax);
        line(nx1, ny1, nx2, ny2);
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    int x1, y1, x2, y2;
    cout << "Enter line endpoints (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;

    line(x1, y1, x2, y2); // Original line
    delay(1000);
    cleardevice();

    liangBarskyClip(x1, y1, x2, y2);
    getch();
    closegraph();
    return 0;
}
```

Original Code - 4. Sutherland-Hodgman Polygon Clipping

```
#include <graphics.h>
#include <iostream>
#include <vector>
using namespace std;

int Xmin = 100, Ymin = 100, Xmax = 400, Ymax = 300;

struct Point { int x, y; };

Point intersect(Point p1, Point p2, int edge) {
    Point i;
    float m = (p2.y - p1.y) / (float)(p2.x - p1.x);
    switch (edge) {
    case 1: // left
        i.x = Xmin; i.y = p1.y + m * (Xmin - p1.x); break;
    case 2: // right
        i.x = Xmax; i.y = p1.y + m * (Xmax - p1.x); break;
    case 3: // bottom
        i.y = Ymin; i.x = p1.x + (Ymin - p1.y) / m; break;
    case 4: // top
        i.y = Ymax; i.x = p1.x + (Ymax - p1.y) / m; break;
    }
    return i;
}

bool inside(Point p, int edge) {
    switch (edge) {
    case 1: return p.x >= Xmin;
    case 2: return p.x <= Xmax;
    case 3: return p.y >= Ymin;
    case 4: return p.y <= Ymax;
    }
    return false;
}

vector<Point> clipPolygon(vector<Point> poly, int edge) {
    vector<Point> out;
    for (int i = 0; i < poly.size(); i++) {
        Point curr = poly[i];
        Point prev = poly[(i + poly.size() - 1) % poly.size()];
        bool currIn = inside(curr, edge);
        bool prevIn = inside(prev, edge);

        if (prevIn && currIn) out.push_back(curr);
        else if (prevIn && !currIn) out.push_back(intersect(prev, curr, edge));
        else if (!prevIn && currIn) {
            out.push_back(intersect(prev, curr, edge));
            out.push_back(curr);
        }
    }
    return out;
}

int main() {

    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    int n;
    cout << "Enter number of polygon vertices: ";
    cin >> n;
    vector<Point> poly(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter vertex " << i + 1 << " (x y): ";
        cin >> poly[i].x >> poly[i].y;
    }
    // Draw original polygon
    for (int i = 0; i < n; i++)
        line(poly[i].x, poly[i].y, poly[(i + 1) % n].x, poly[(i + 1) % n].y);

    delay(1500);
    cleardevice();

    rectangle(Xmin, Ymin, Xmax, Ymax);

    for (int edge = 1; edge <= 4; edge++)
        poly = clipPolygon(poly, edge);

    for (int i = 0; i < poly.size(); i++)
        line(poly[i].x, poly[i].y, poly[(i + 1) % poly.size()].x, poly[(i + 1) % poly.size()].y);
```

```
        getch();
        closegraph();

        return 0;
    }
```

Original Output



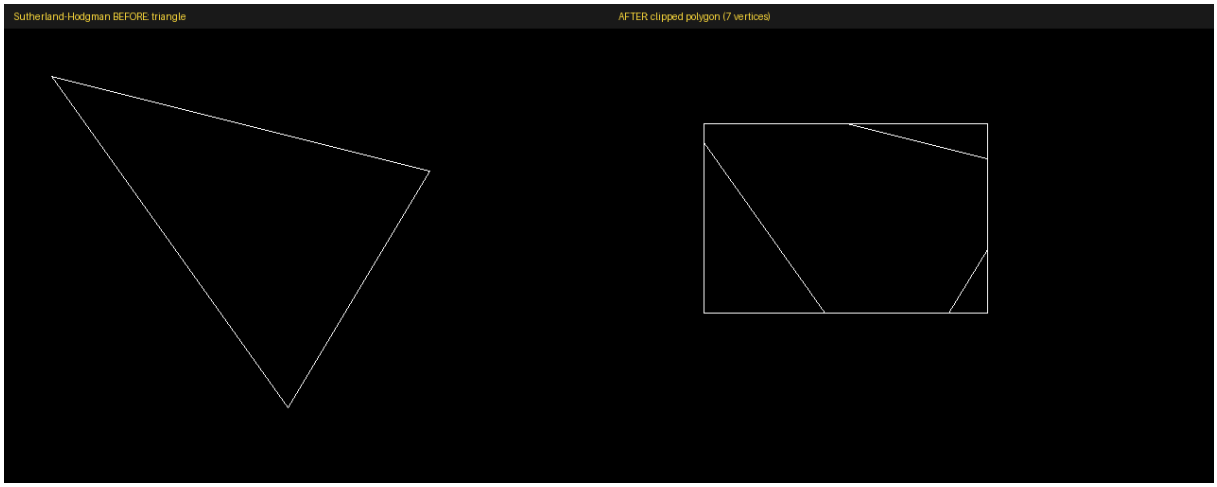
Point clipping: inside point green, outside point red



Cohen-Sutherland: before and after clipping



Liang-Barsky: before and after clipping



Sutherland-Hodgman: triangle before and after clipping

Corrected Code (matches theory: window (-100,-100)-(100,100), grey original, red clipped) - 1. Point

```
// Practical 5.1 (theory version): Point clipping.
// Clipping window in WORLD coordinates: (xmin,ymin)=(-100,-100), (xmax,ymax)=(100,100).
// World -> screen: 2 pixels per unit, origin at screen centre (320,240).
#include <graphics.h>
#include <iostream>
using namespace std;

const int WXMIN = -100, WYMIN = -100, WXMAX = 100, WYMAX = 100;
int mx(int x) { return 320 + 2 * x; }
int my(int y) { return 240 - 2 * y; }

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    setcolor(WHITE);
    rectangle(mx(WXMIN), my(WYMAX), mx(WXMAX), my(WYMIN)); // clipping window
    outtextxy(10, 10, "Point clipping: window (-100,-100)-(100,100)");

    int x, y;
    cout << "Enter point (x y) in world coordinates: ";
    cin >> x >> y;
    if (x >= WXMIN && x <= WXMAX && y >= WYMIN && y <= WYMAX) {
        putpixel(mx(x), my(y), RED); // visible part in red
        cout << "Point (" << x << "," << y << ") is inside: drawn in red.\n";
    } else {
        cout << "Point (" << x << "," << y << ") rejected: outside window.\n";
    }
    getch();
    closegraph();
    return 0;
}
```

Corrected Code - 2. Cohen-Sutherland

```
// Practical 5.2 (theory version): Cohen-Sutherland line clipping.
// Clipping window in WORLD coordinates: (-100,-100)-(100,100).
// Original line in grey, clipped (visible) part in red.
#include <graphics.h>
#include <iostream>
using namespace std;

const int INSIDE = 0; // 0000
const int LEFT = 1;   // 0001
const int RIGHT = 2;  // 0010
const int BOTTOM = 4; // 0100
const int TOP = 8;    // 1000

const int WXMIN = -100, WYMIN = -100, WXMAX = 100, WYMAX = 100;
int mx(int x) { return 320 + 2 * x; }
int my(int y) { return 240 - 2 * y; }

int computeCode(int x, int y) {
    int code = INSIDE;
    if (x < WXMIN) code |= LEFT;
    else if (x > WXMAX) code |= RIGHT;
    if (y < WYMIN) code |= BOTTOM;
    else if (y > WYMAX) code |= TOP;
    return code;
}

// Clips in world coordinates; draws window (white) + visible part (red).
void cohenSutherlandClip(int x1, int y1, int x2, int y2) {
    int code1 = computeCode(x1, y1);
    int code2 = computeCode(x2, y2);
    bool accept = false;

    while (true) {
        if ((code1 | code2) == 0) { accept = true; break; }      // trivial accept
        else if (code1 & code2) break;                            // trivial reject
        else {
            int codeOut = code1 ? code1 : code2;
            int x, y;
            if (codeOut & TOP) {
                x = x1 + (x2 - x1) * (WYMAX - y1) / (y2 - y1); y = WYMAX;
            } else if (codeOut & BOTTOM) {
                x = x1 + (x2 - x1) * (WYMIN - y1) / (y2 - y1); y = WYMIN;
            } else if (codeOut & RIGHT) {
                y = y1 + (y2 - y1) * (WXMAX - x1) / (x2 - x1); x = WXMAX;
            } else {
                y = y1 + (y2 - y1) * (WXMIN - x1) / (x2 - x1); x = WXMIN;
            }
            if (codeOut == code1) { x1 = x; y1 = y; code1 = computeCode(x1, y1); }
            else { x2 = x; y2 = y; code2 = computeCode(x2, y2); }
        }
    }
    setcolor(WHITE);
    rectangle(mx(WXMIN), my(WYMAX), mx(WXMAX), my(WYMIN));
    if (accept) {

        cout << "Clipped line: (" << x1 << "," << y1 << ")-(" << x2 << "," << y2 << ")\n";
        setcolor(RED);
        line(mx(x1), my(y1), mx(x2), my(y2));
    } else {
        cout << "Line rejected: completely outside window.\n";
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    int x1, y1, x2, y2;
    cout << "Enter line endpoints in world coordinates (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;

    setcolor(LIGHTGRAY);
    line(mx(x1), my(y1), mx(x2), my(y2)); // original in grey
    outtextxy(10, 10, "Cohen-Sutherland: grey=original, red=clipped");
    delay(1000);
    cleardevice();
    cohenSutherlandClip(x1, y1, x2, y2);
    getch();
    closegraph();
    return 0;
```

```
}
```

Corrected Code - 3. Liang-Barsky

```
// Practical 5.3 (theory version): Liang-Barsky line clipping.
// Clipping window in WORLD coordinates: (-100,-100)-(100,100).
// Original line in grey, clipped (visible) part in red.
#include <graphics.h>
#include <iostream>
using namespace std;

const int WXMIN = -100, WYMIN = -100, WXMAX = 100, WYMAX = 100;
int mx(int x) { return 320 + 2 * x; }
int my(int y) { return 240 - 2 * y; }

bool clipTest(float p, float q, float &u1, float &u2) {
    if (p == 0 && q < 0) return false; // parallel and outside
    float r = q / p;
    if (p < 0) {
        if (r > u2) return false;
        else if (r > u1) u1 = r;
    } else if (p > 0) {
        if (r < u1) return false;
        else if (r < u2) u2 = r;
    }
    return true;
}

void liangBarskyClip(int x1, int y1, int x2, int y2) {
    float dx = x2 - x1, dy = y2 - y1;
    float u1 = 0.0, u2 = 1.0;
    setcolor(WHITE);
    rectangle(mx(WXMIN), my(WYMAX), mx(WXMAX), my(WYMIN));
    if (clipTest(-dx, x1 - WXMIN, u1, u2) &&
        clipTest(dx, WXMAX - x1, u1, u2) &&
        clipTest(-dy, y1 - WYMIN, u1, u2) &&
        clipTest(dy, WYMAX - y1, u1, u2)) {
        int nx1 = x1 + u1 * dx, ny1 = y1 + u1 * dy;
        int nx2 = x1 + u2 * dx, ny2 = y1 + u2 * dy;
        cout << "Clipped line: (" << nx1 << "," << ny1 << ")-(" << nx2 << "," << ny2 << ")\n";
        setcolor(RED);
        line(mx(nx1), my(ny1), mx(nx2), my(ny2));
    } else {
        cout << "Line rejected: completely outside window.\n";
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    int x1, y1, x2, y2;
    cout << "Enter line endpoints in world coordinates (x1 y1 x2 y2): ";
    cin >> x1 >> y1 >> x2 >> y2;

    setcolor(LIGHTGRAY);
    line(mx(x1), my(y1), mx(x2), my(y2)); // original in grey
    outtextxy(10, 10, "Liang-Barsky: grey=original, red=clipped");

    delay(1000);
    cleardevice();
    liangBarskyClip(x1, y1, x2, y2);
    getch();
    closegraph();
    return 0;
}
```

Corrected Code - 4. Sutherland-Hodgman

```
// Practical 5.4 (theory version): Sutherland-Hodgman polygon clipping.
// Clipping window in WORLD coordinates: (-100,-100)-(100,100).
// Original polygon in grey, clipped (visible) part in red.
#include <graphics.h>
#include <iostream>
#include <vector>
using namespace std;

const int WXMIN = -100, WYMIN = -100, WXMAX = 100, WYMAX = 100;
int mx(int x) { return 320 + 2 * x; }
int my(int y) { return 240 - 2 * y; }

struct Point { int x, y; };

// edge: 1=left 2=right 3=bottom 4=top
Point intersectPt(Point p1, Point p2, int edge) {
    Point i;
    float m = (p2.y - p1.y) / (float)(p2.x - p1.x);
    switch (edge) {
    case 1: i.x = WXMIN; i.y = p1.y + m * (WXMIN - p1.x); break;
    case 2: i.x = WXMAX; i.y = p1.y + m * (WXMAX - p1.x); break;
    case 3: i.y = WYMIN; i.x = p1.x + (WYMIN - p1.y) / m; break;
    case 4: i.y = WYMAX; i.x = p1.x + (WYMAX - p1.y) / m; break;
    }
    return i;
}

bool insidePt(Point p, int edge) {
    switch (edge) {
    case 1: return p.x >= WXMIN;
    case 2: return p.x <= WXMAX;
    case 3: return p.y >= WYMIN;
    case 4: return p.y <= WYMAX;
    }
    return false;
}

vector<Point> clipPolygon(vector<Point> poly, int edge) {
    vector<Point> out;
    for (size_t i = 0; i < poly.size(); i++) {
        Point curr = poly[i];
        Point prev = poly[(i + poly.size() - 1) % poly.size()];
        bool currIn = insidePt(curr, edge), prevIn = insidePt(prev, edge);
        if (prevIn && currIn) out.push_back(curr);
        else if (prevIn && !currIn) out.push_back(intersectPt(prev, curr, edge));
        else if (!prevIn && currIn) {
            out.push_back(intersectPt(prev, curr, edge));
            out.push_back(curr);
        }
    }
    return out;
}

void drawPolyWorld(const vector<Point> &poly, int color) {

    setcolor(color);
    for (size_t i = 0; i < poly.size(); i++)
        line(mx(poly[i].x), my(poly[i].y),
             mx(poly[(i + 1) % poly.size()].x), my(poly[(i + 1) % poly.size()].y));
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

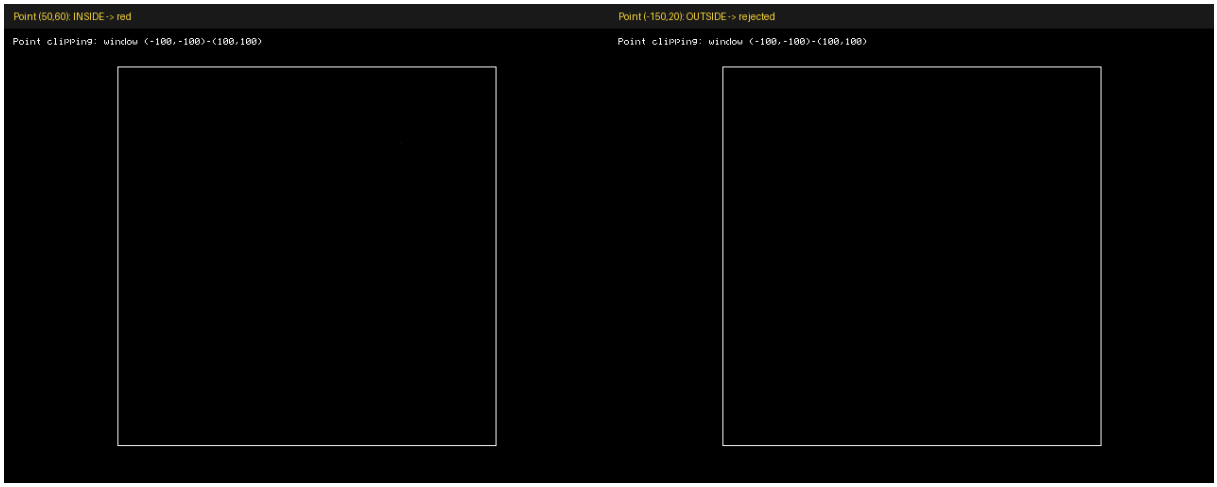
    int n;
    cout << "Enter number of polygon vertices: ";
    cin >> n;
    vector<Point> poly(n);
    cout << "Enter vertices in world coordinates (x y):\n";
    for (int i = 0; i < n; i++) cin >> poly[i].x >> poly[i].y;

    drawPolyWorld(poly, LIGHTGRAY); // original in grey
    outtextxy(10, 10, "Sutherland-Hodgman: grey=original, red=clipped");
    delay(1500);
    cleardevice();

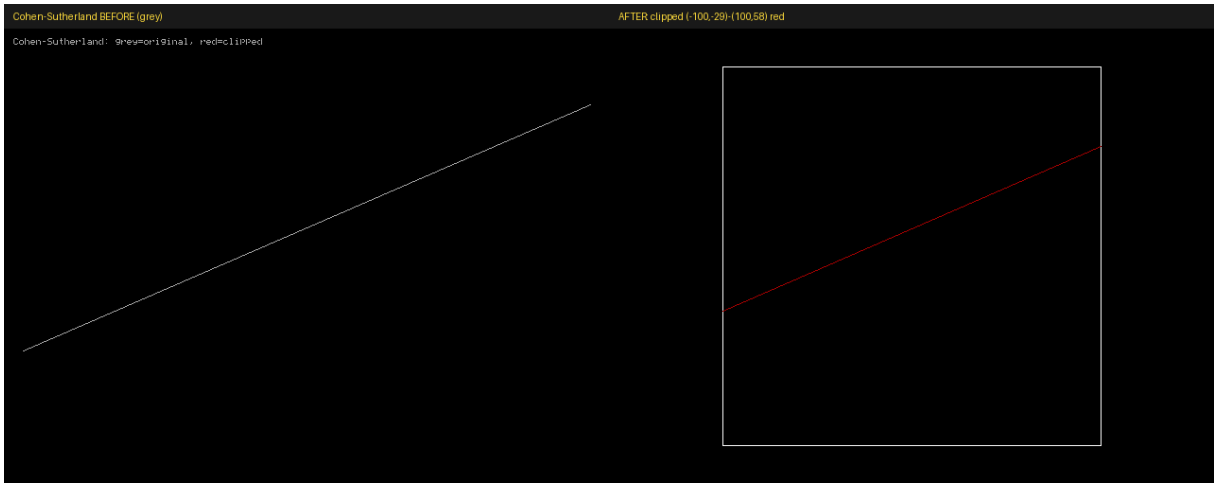
    setcolor(WHITE);
    rectangle(mx(WXMIN), my(WYMAX), mx(WXMAX), my(WYMIN));
    for (int edge = 1; edge <= 4; edge++) poly = clipPolygon(poly, edge);
```

```
    cout << "Clipped polygon has " << poly.size() << " vertices:\n";
    for (auto &p : poly) cout << "(" << p.x << "," << p.y << ") ";
    cout << "\n";
    drawPolyWorld(poly, RED); // visible part in red
    getch();
    closegraph();
    return 0;
}
```

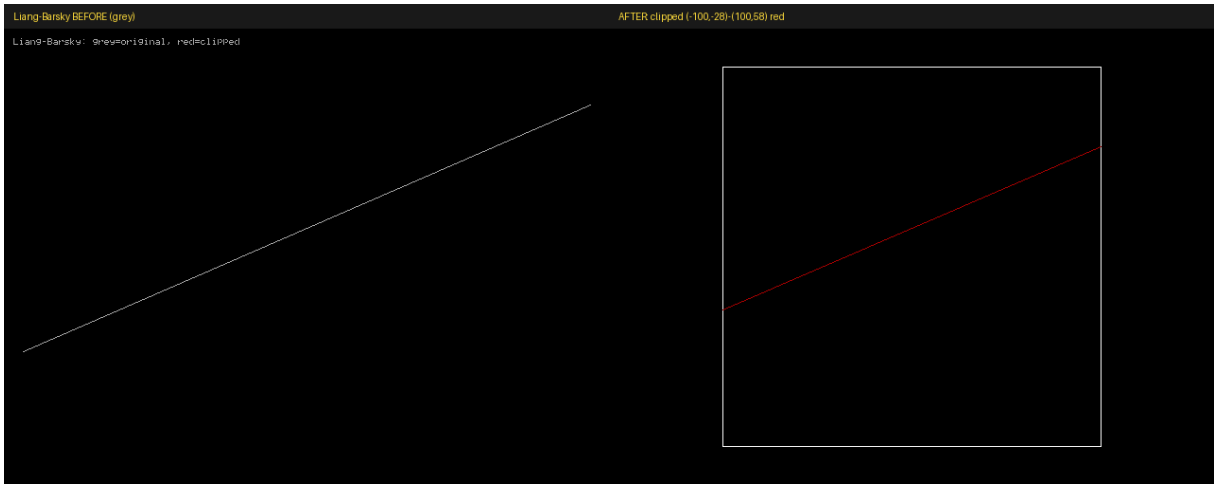
Corrected Output



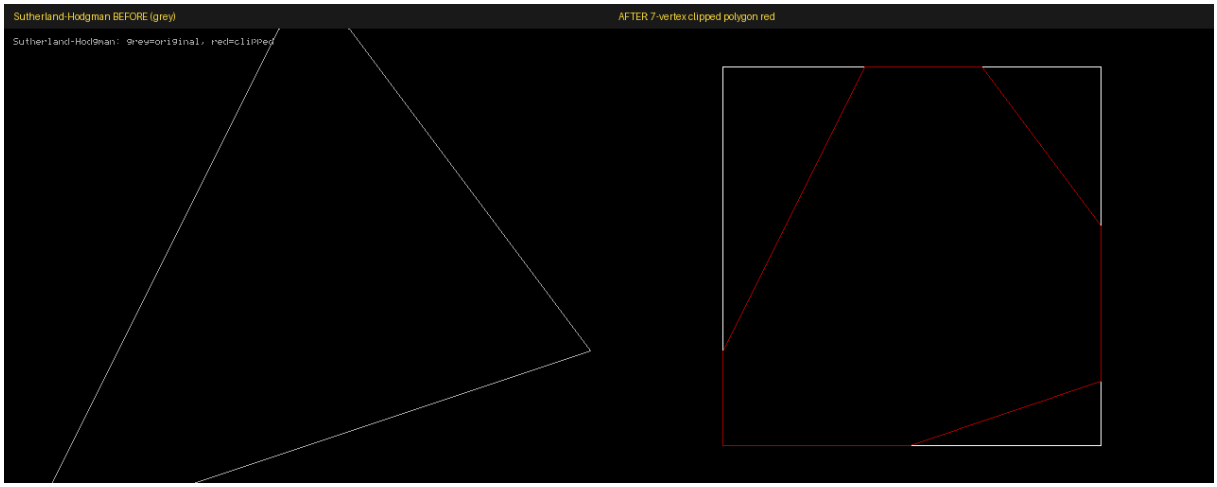
Point clipping in world coordinates



Cohen-Sutherland in world coordinates



Liang-Barsky in world coordinates



Sutherland-Hodgman in world coordinates

Practical No. 6 - Polygon Clipping with Viewport (Mouse + Keyboard)

Original Code

```
#include <graphics.h>
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

const int TOP = 8;
const int BOTTOM = 4;
const int RIGHT = 2;
const int LEFT = 1;

int computeCode(int x, int y, int xmin, int ymin, int xmax, int ymax) {
    int code = 0;
    if (y > ymax) code |= TOP;
    if (y < ymin) code |= BOTTOM;
    if (x > xmax) code |= RIGHT;
    if (x < xmin) code |= LEFT;
    return code;
}

bool cohenSutherlandClip(int x1, int y1, int x2, int y2,
                         int xmin, int ymin, int xmax, int ymax,
                         int &nx1, int &ny1, int &nx2, int &ny2) {
    int code1 = computeCode(x1, y1, xmin, ymin, xmax, ymax);
    int code2 = computeCode(x2, y2, xmin, ymin, xmax, ymax);
    bool accept = false;
    double dx, dy;
    while (true) {
        if ((code1 | code2) == 0) {
            accept = true;
            nx1 = x1; ny1 = y1;
            nx2 = x2; ny2 = y2;
            break;
        } else if (code1 & code2) {
            break;
        } else {
            int outCode = code1 ? code1 : code2;
            double x = 0.0, y = 0.0;
            if (outCode & TOP) {
                dx = x2 - x1; dy = y2 - y1;
                if (fabs(dy) < 1e-9) break;
                x = x1 + dx * (double)(ymax - y1) / dy;
                y = ymax;
            } else if (outCode & BOTTOM) {
                dx = x2 - x1; dy = y2 - y1;
                if (fabs(dy) < 1e-9) break;
                x = x1 + dx * (double)(ymin - y1) / dy;
                y = ymin;
            } else if (outCode & RIGHT) {
                dx = x2 - x1; dy = y2 - y1;
                if (fabs(dx) < 1e-9) break;
                y = y1 + dy * (double)(xmax - x1) / dx;
                x = xmax;
            } else if (outCode & LEFT) {

                dx = x2 - x1; dy = y2 - y1;
                if (fabs(dx) < 1e-9) break;
                y = y1 + dy * (double)(xmin - x1) / dx;
                x = xmin;
            }
            if (outCode == code1) {
                x1 = (int)round(x);
                y1 = (int)round(y);
                code1 = computeCode(x1, y1, xmin, ymin, xmax, ymax);
            } else {
                x2 = (int)round(x);
                y2 = (int)round(y);
                code2 = computeCode(x2, y2, xmin, ymin, xmax, ymax);
            }
        }
    }
    return accept;
}
```

```
int main() {
    int n;
    cout << "Enter number of polygon vertices: ";
    if (!(cin >> n) || n < 2) return 1;
    vector<pair<int,int>> pts(n);
    cout << "Enter coordinates as x y pairs:\n";
    for (int i = 0; i < n; ++i) cin >> pts[i].first >> pts[i].second;

    int xmin = 150, ymin = 100, xmax = 400, ymax = 300;
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);

    setcolor(RED);
    rectangle(xmin, ymin, xmax, ymax);

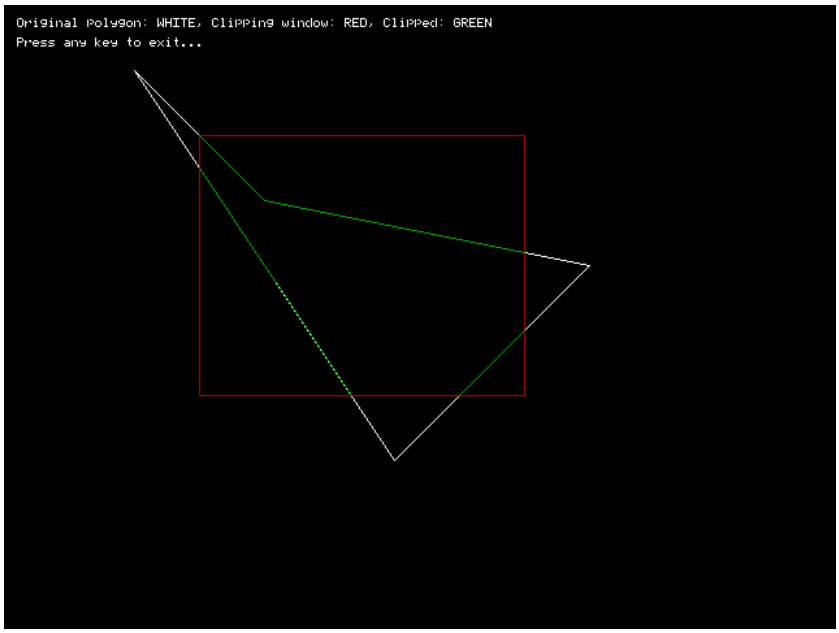
    setcolor(WHITE);
    for (int i = 0; i < n; ++i) {
        int x1 = pts[i].first, y1 = pts[i].second;
        int x2 = pts[(i+1)%n].first, y2 = pts[(i+1)%n].second;
        line(x1, y1, x2, y2);
    }
    delay(800);

    setcolor(GREEN);
    for (int i = 0; i < n; ++i) {
        int x1 = pts[i].first, y1 = pts[i].second;
        int x2 = pts[(i+1)%n].first, y2 = pts[(i+1)%n].second;
        int cx1, cy1, cx2, cy2;
        if (cohenSutherlandClip(x1, y1, x2, y2, xmin, ymin, xmax, ymax, cx1, cy1, cx2, cy2)) {
            line(cx1, cy1, cx2, cy2);
        }
    }

    setcolor(WHITE);
    outtextxy(10, 10, "Original polygon: WHITE, Clipping window: RED, Clipped: GREEN");

    outtextxy(10, 25, "Press any key to exit...");
    getch();
    closegraph();
    return 0;
}
```

Original Output



Edge-by-edge Cohen-Sutherland polygon clipping (keyboard input)

Corrected Code (matches theory: mouse input, 800x400, viewport, keyboard controls)

```
// Practical 6 (theory version): Sutherland-Hodgman polygon clipping with
// mouse + keyboard interface and window-to-viewport mapping.
// 800x400 screen: left half = window area, right half = viewport.
//   Left click: add polygon vertex | Right click: close polygon (>=3 vertices)
//   c: clip | r: reset | Arrow keys: move window 10px (re-clips) | Esc: exit
#include <graphics.h>
#include <iostream>
#include <vector>
using namespace std;

struct Pt { int x, y; };

// clipping window inside the left half (moves with arrow keys)
int WXL = 50, WYL = 50, WXR = 350, WYR = 350;
// viewport inside the right half (fixed)
const int VXL = 450, VYL = 50, VXR = 750, VYR = 350;

vector<Pt> poly;      // entered polygon
vector<Pt> clipped;   // clipped polygon
bool closed = false;

bool insidePt(Pt p, int edge) {
    switch (edge) {
    case 1: return p.x >= WXL;
    case 2: return p.x <= WXR;
    case 3: return p.y >= WYL;
    case 4: return p.y <= WYR;
    }
    return false;
}

Pt intersectPt(Pt p1, Pt p2, int edge) {
    Pt i;
    float m = (p2.y - p1.y) / (float)(p2.x - p1.x);
    switch (edge) {
    case 1: i.x = WXL; i.y = p1.y + m * (WXL - p1.x); break;
    case 2: i.x = WXR; i.y = p1.y + m * (WXR - p1.x); break;
    case 3: i.y = WYL; i.x = p1.x + (WYL - p1.y) / m; break;
    case 4: i.y = WYR; i.x = p1.x + (WYR - p1.y) / m; break;
    }
    return i;
}

vector<Pt> clipAgainst(vector<Pt> in, int edge) {
    vector<Pt> out;
    for (size_t i = 0; i < in.size(); i++) {
        Pt curr = in[i], prev = in[(i + in.size() - 1) % in.size()];
        bool ci = insidePt(curr, edge), pi = insidePt(prev, edge);
        if (pi && ci) out.push_back(curr);
        else if (pi && !ci) out.push_back(intersectPt(prev, curr, edge));
        else if (!pi && ci) {
            out.push_back(intersectPt(prev, curr, edge));
            out.push_back(curr);
        }

    }
    return out;
}

void doClip() {
    clipped = poly;
    for (int e = 1; e <= 4; e++) clipped = clipAgainst(clipped, e);
}

// window -> viewport transformation
Pt w2v(Pt p) {
    Pt v;
    v.x = VXL + (p.x - WXL) * (VXR - VXL) / (WXR - WXL);
    v.y = VYL + (p.y - WYL) * (VYR - VYL) / (WYR - WYL);
    return v;
}

void drawPolyPts(const vector<Pt> &v, bool close, int color) {
    if (v.empty()) return;
    setcolor(color);
    for (size_t i = 0; i + 1 < v.size(); i++)
        line(v[i].x, v[i].y, v[i + 1].x, v[i + 1].y);
    if (close && v.size() > 1)
```

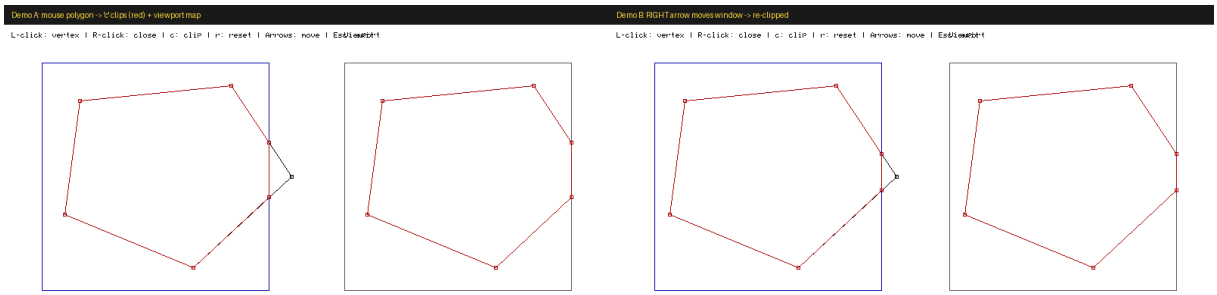
```
        line(v.back().x, v.back().y, v[0].x, v[0].y);
    for (auto &p : v) rectangle(p.x - 2, p.y - 2, p.x + 2, p.y + 2); // vertex markers
}

void redraw() {
    setbkcolor(WHITE);
    cleardevice();
    setcolor(BLUE);
    rectangle(WXL, WYL, WXR, WYR);   // clipping window
    setcolor(DARKGRAY);
    rectangle(VXL, VYL, VXR, VYR);   // viewport border
    setcolor(BLACK);
    outtextxy(10, 10, "L-click: vertex | R-click: close | c: clip | r: reset | Arrows: move | Esc: exit");
    outtextxy(450, 10, "Viewport");
    drawPolyPts(poly, closed, BLACK); // original polygon in black
    if (closed && !clipped.empty()) {
        drawPolyPts(clipped, true, RED); // clipped polygon in red (window)
        vector<Pt> vp;
        for (auto &p : clipped) vp.push_back(w2v(p));
        drawPolyPts(vp, true, RED);      // mapped into viewport
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);
    initmouse();
    showmouseptr();
    cout << "Draw the polygon with the mouse in the graphics window.\n";
    redraw();
    while (true) {

        if (ismouseclick(WM_LBUTTONDOWN)) {
            int x, y;
            getmouseclick(WM_LBUTTONDOWN, x, y);
            if (!closed && x < 400) { poly.push_back({x, y}); redraw(); }
        }
        if (ismouseclick(WM_RBUTTONDOWN)) {
            int x, y;
            getmouseclick(WM_RBUTTONDOWN, x, y);
            if (!closed && poly.size() >= 3) { closed = true; redraw(); }
        }
        if (kbhit()) {
            int ch = getch();
            if (ch == 0) ch = getch(); // extended key (arrows)
            if (ch == 'c' || ch == 'C') { if (closed) { doClip(); redraw(); } }
            else if (ch == 'r' || ch == 'R') {
                poly.clear(); clipped.clear(); closed = false; redraw();
            }
            else if (ch == 75) { WXL -= 10; WXR -= 10; if (closed) doClip(); redraw(); }
            else if (ch == 77) { WXL += 10; WXR += 10; if (closed) doClip(); redraw(); }
            else if (ch == 72) { WYL -= 10; WYR -= 10; if (closed) doClip(); redraw(); }
            else if (ch == 80) { WYL += 10; WYR += 10; if (closed) doClip(); redraw(); }
            else if (ch == 27) break; // Esc
        }
    }
    closegraph();
    return 0;
}
```

Corrected Output



Left: mouse polygon clipped with 'c' + viewport map. Right: arrow key moves window, re-clips.

Practical No. 7 - 2-D and 3-D Transformations

Original Code (2-D only)

```
#include <graphics.h>
#include <iostream>
#include <conio.h>
#include <math.h>
using namespace std;

void drawTriangle(int x1, int y1, int x2, int y2, int x3, int y3, int color) {
    setcolor(color);
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}

void performAllTransformations() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int x1 = 100, y1 = 200;
    int x2 = 200, y2 = 200;
    int x3 = 150, y3 = 100;
    int tx, ty;
    float sx, sy, angle;

    cout << "\n========================================" << endl;
    cout << " 2D TRANSFORMATIONS - ALL OPERATIONS " << endl;
    cout << "========================================\n" << endl;
    cout << "Enter Translation factors (tx ty): ";
    cin >> tx >> ty;

    cout << "Enter Scaling factors (sx sy): ";
    cin >> sx >> sy;

    cout << "Enter Rotation angle (degrees): ";
    cin >> angle;

    cout << "\n========================================\n" << endl;

    // STEP 1: Draw Original Triangle
    cout << "STEP 1: ORIGINAL TRIANGLE" << endl;
    cout << "Coordinates: (" << x1 << "," << y1 << "), ("
         << x2 << "," << y2 << "), (" << x3 << "," << y3 << ")" << endl;

    setcolor(WHITE);
    outtextxy(10, 10, "WHITE = Original");
    drawTriangle(x1, y1, x2, y2, x3, y3, WHITE);
    outtextxy(x1-20, y1+20, "Original");

    delay(1500);

    // STEP 2: Translation
    cout << "\nSTEP 2: TRANSLATION" << endl;
    cout << "Translation factors: tx = " << tx << ", ty = " << ty << endl;

    int tx1 = x1 + tx;
    int ty1 = y1 + ty;

    int tx2 = x2 + tx;
    int ty2 = y2 + ty;
    int tx3 = x3 + tx;
    int ty3 = y3 + ty;

    cout << "After Translation: (" << tx1 << "," << ty1 << "), ("
         << tx2 << "," << ty2 << "), (" << tx3 << "," << ty3 << ")" << endl;

    setcolor(GREEN);
    outtextxy(10, 30, "GREEN = After Translation");
    drawTriangle(tx1, ty1, tx2, ty2, tx3, ty3, GREEN);
    outtextxy(tx1-20, ty1+20, "Translated");

    delay(1500);

    // STEP 3: Scaling (applied on translated coordinates)
    cout << "\nSTEP 3: SCALING (on translated triangle)" << endl;
    cout << "Scaling factors: sx = " << sx << ", sy = " << sy << endl;

    // Scale from the center of the translated triangle
    int centerX = (tx1 + tx2 + tx3) / 3;
```

```
    int centerY = (ty1 + ty2 + ty3) / 3;

    int sx1 = centerX + (tx1 - centerX) * sx;
    int sy1 = centerY + (ty1 - centerY) * sy;
    int sx2 = centerX + (tx2 - centerX) * sx;
    int sy2 = centerY + (ty2 - centerY) * sy;
    int sx3 = centerX + (tx3 - centerX) * sx;
    int sy3 = centerY + (ty3 - centerY) * sy;

    cout << "After Scaling: (" << sx1 << "," << sy1 << "), ("
         << sx2 << "," << sy2 << "), (" << sx3 << "," << sy3 << ")" << endl;

    setcolor(YELLOW);
    outtextxy(10, 50, "YELLOW = After Scaling");
    drawTriangle(sx1, sy1, sx2, sy2, sx3, sy3, YELLOW);
    outtextxy(sx1-20, sy1+30, "Scaled");

    delay(1500);

    // STEP 4: Rotation (applied on scaled coordinates)
    cout << "\nSTEP 4: ROTATION (on scaled triangle)" << endl;
    cout << "Rotation angle: " << angle << " degrees" << endl;

    // Convert angle to radians
    float radian = angle * 3.14159 / 180.0;
    float cosTheta = cos(radian);
    float sinTheta = sin(radian);

    // Rotate around the center
    int rx1 = centerX + (sx1 - centerX) * cosTheta - (sy1 - centerY) * sinTheta;
    int ry1 = centerY + (sx1 - centerX) * sinTheta + (sy1 - centerY) * cosTheta;
    int rx2 = centerX + (sx2 - centerX) * cosTheta - (sy2 - centerY) * sinTheta;
    int ry2 = centerY + (sx2 - centerX) * sinTheta + (sy2 - centerY) * cosTheta;

    int rx3 = centerX + (sx3 - centerX) * cosTheta - (sy3 - centerY) * sinTheta;
    int ry3 = centerY + (sx3 - centerX) * sinTheta + (sy3 - centerY) * cosTheta;

    cout << "After Rotation: (" << rx1 << "," << ry1 << "), ("
         << rx2 << "," << ry2 << "), (" << rx3 << "," << ry3 << ")" << endl;

    setcolor(CYAN);
    outtextxy(10, 70, "CYAN = After Rotation");
    drawTriangle(rx1, ry1, rx2, ry2, rx3, ry3, CYAN);
    outtextxy(rx1-20, ry1+30, "Rotated");

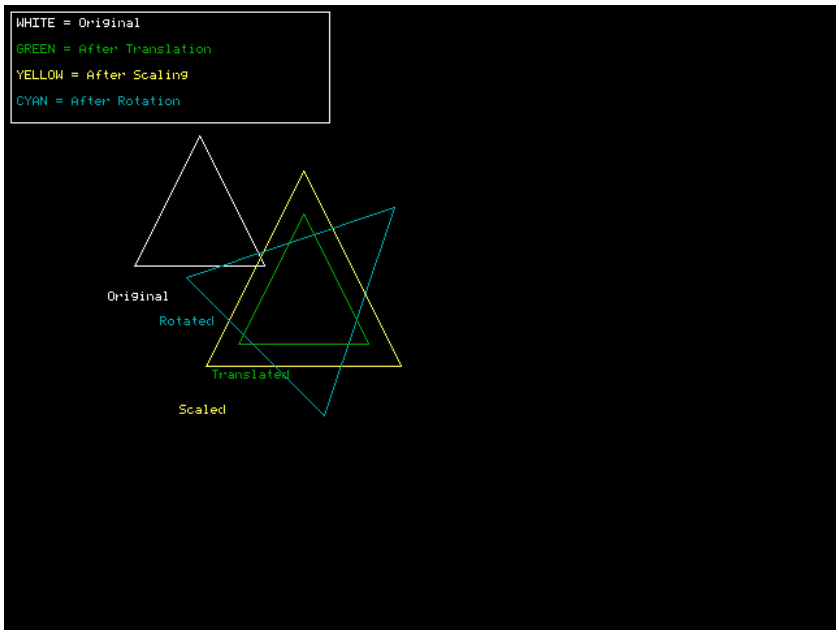
    cout << "\n========================================" << endl;
    cout << "All transformations completed!" << endl;
    cout << "Press any key to exit..." << endl;
    cout << "========================================\n" << endl;

    setcolor(WHITE);
    rectangle(5, 5, 250, 90);

    getch();
    closegraph();
}

int main() {
    cout << "\n*** 2D TRANSFORMATIONS - COMBINED OPERATIONS ***\n" << endl;
    cout << "This program will apply:" << endl;
    cout << "1. Translation" << endl;
    cout << "2. Scaling" << endl;
    cout << "3. Rotation" << endl;
    cout << "on the same triangle sequentially.\n" << endl;
    performAllTransformations();
    return 0;
}
```

Original Output



2-D triangle: original, translated, scaled, rotated

Corrected Code (matches theory: 2-D + 3-D cube with 4x4 matrices, oblique projection)

```
// Practical 7 (theory version): 2-D and 3-D transformations.
// 2-D: translation, scaling, rotation of a triangle (sequential, as before).
// 3-D: translation, scaling, rotation about X/Y/Z of a cube (side 100 at origin)
//      using 4x4 homogeneous matrices, drawn with oblique (cabinet) projection:
//      xp = x + 0.5*z*cos45, yp = y + 0.5*z*sin45. Original blue, transformed red.
#include <graphics.h>
#include <iostream>
#include <conio.h>
#include <math.h>
using namespace std;

// ============================ 2-D part ============================
void drawTriangle(int x1, int y1, int x2, int y2, int x3, int y3, int color) {
    setcolor(color);
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}

void transformations2D() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int x1 = 100, y1 = 200, x2 = 200, y2 = 200, x3 = 150, y3 = 100;
    int tx, ty;
    float sx, sy, angle;

    cout << "Enter Translation factors (tx ty): ";
    cin >> tx >> ty;
    cout << "Enter Scaling factors (sx sy): ";
    cin >> sx >> sy;
    cout << "Enter Rotation angle (degrees): ";
    cin >> angle;

    setcolor(WHITE);
    outtextxy(10, 10, "WHITE = Original");
    drawTriangle(x1, y1, x2, y2, x3, y3, WHITE);
    outtextxy(x1 - 20, y1 + 20, "Original");
    cout << "Original: (" << x1 << "," << y1 << "), (" << x2 << "," << y2
         << "), (" << x3 << "," << y3 << ")\n";
    delay(1500);

    int tx1 = x1 + tx, ty1 = y1 + ty;
    int tx2 = x2 + tx, ty2 = y2 + ty;
    int tx3 = x3 + tx, ty3 = y3 + ty;
    cout << "After Translation: (" << tx1 << "," << ty1 << "), (" << tx2 << "," << ty2
         << "), (" << tx3 << "," << ty3 << ")\n";
    setcolor(GREEN);
    outtextxy(10, 30, "GREEN = After Translation");
    drawTriangle(tx1, ty1, tx2, ty2, tx3, ty3, GREEN);
    delay(1500);

    int centerX = (tx1 + tx2 + tx3) / 3, centerY = (ty1 + ty2 + ty3) / 3;
    int sx1 = centerX + (tx1 - centerX) * sx, sy1 = centerY + (ty1 - centerY) * sy;
    int sx2 = centerX + (tx2 - centerX) * sx, sy2 = centerY + (ty2 - centerY) * sy;

    int sx3 = centerX + (tx3 - centerX) * sx, sy3 = centerY + (ty3 - centerY) * sy;
    cout << "After Scaling: (" << sx1 << "," << sy1 << "), (" << sx2 << "," << sy2
         << "), (" << sx3 << "," << sy3 << ")\n";
    setcolor(YELLOW);
    outtextxy(10, 50, "YELLOW = After Scaling");
    drawTriangle(sx1, sy1, sx2, sy2, sx3, sy3, YELLOW);
    delay(1500);

    float radian = angle * 3.14159 / 180.0;
    float c = cos(radian), s = sin(radian);
    int rx1 = centerX + (sx1 - centerX) * c - (sy1 - centerY) * s;
    int ry1 = centerY + (sx1 - centerX) * s + (sy1 - centerY) * c;
    int rx2 = centerX + (sx2 - centerX) * c - (sy2 - centerY) * s;
    int ry2 = centerY + (sx2 - centerX) * s + (sy2 - centerY) * c;
    int rx3 = centerX + (sx3 - centerX) * c - (sy3 - centerY) * s;
    int ry3 = centerY + (sx3 - centerX) * s + (sy3 - centerY) * c;
    cout << "After Rotation: (" << rx1 << "," << ry1 << "), (" << rx2 << "," << ry2
         << "), (" << rx3 << "," << ry3 << ")\n";
    setcolor(CYAN);
    outtextxy(10, 70, "CYAN = After Rotation");
    drawTriangle(rx1, ry1, rx2, ry2, rx3, ry3, CYAN);

    setcolor(WHITE);
```

```
    rectangle(5, 5, 250, 90);
    getch();
    closegraph();
}

// ============================ 3-D part ============================
struct Vec3 { float x, y, z; };
float M4[4][4];

void mIdentity() {
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            M4[i][j] = (i == j) ? 1 : 0;
}
void mTranslate(float tx, float ty, float tz) {
    mIdentity(); M4[0][3] = tx; M4[1][3] = ty; M4[2][3] = tz;
}
void mScale(float sx, float sy, float sz) {
    mIdentity(); M4[0][0] = sx; M4[1][1] = sy; M4[2][2] = sz;
}
void mRotX(float deg) {
    float r = deg * 3.14159 / 180.0, c = cos(r), s = sin(r);
    mIdentity(); M4[1][1] = c; M4[1][2] = -s; M4[2][1] = s; M4[2][2] = c;
}
void mRotY(float deg) {
    float r = deg * 3.14159 / 180.0, c = cos(r), s = sin(r);
    mIdentity(); M4[0][0] = c; M4[0][2] = s; M4[2][0] = -s; M4[2][2] = c;
}
void mRotZ(float deg) {
    float r = deg * 3.14159 / 180.0, c = cos(r), s = sin(r);
    mIdentity(); M4[0][0] = c; M4[0][1] = -s; M4[1][0] = s; M4[1][1] = c;

}
Vec3 mApply(Vec3 v) {
    Vec3 r;
    r.x = M4[0][0]*v.x + M4[0][1]*v.y + M4[0][2]*v.z + M4[0][3];
    r.y = M4[1][0]*v.x + M4[1][1]*v.y + M4[1][2]*v.z + M4[1][3];
    r.z = M4[2][0]*v.x + M4[2][1]*v.y + M4[2][2]*v.z + M4[2][3];
    return r;
}
void printMat() {
    cout << "Transformation matrix:\n";
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) cout << M4[i][j] << "\t";
        cout << "\n";
    }
}

Vec3 cube[8] = {
    {0,0,0}, {100,0,0}, {100,100,0}, {0,100,0},
    {0,0,100}, {100,0,100}, {100,100,100}, {0,100,100}
};
int cubeEdges[12][2] = {
    {0,1},{1,2},{2,3},{3,0}, {4,5},{5,6},{6,7},{7,4}, {0,4},{1,5},{2,6},{3,7}
};

// oblique (cabinet) projection, drawn with screen offset (ox,oy)
void drawCube(Vec3 v[8], int color, int ox, int oy) {
    setcolor(color);
    const float k = 0.5 * 0.70710678f; // 0.5 * cos45 = 0.5 * sin45
    for (int e = 0; e < 12; e++) {
        Vec3 a = v[cubeEdges[e][0]], b = v[cubeEdges[e][1]];
        int x1 = ox + (int)(a.x + k * a.z + 0.5f);
        int y1 = oy + (int)(a.y + k * a.z + 0.5f);
        int x2 = ox + (int)(b.x + k * b.z + 0.5f);
        int y2 = oy + (int)(b.y + k * b.z + 0.5f);
        line(x1, y1, x2, y2);
    }
}

void transformations3D() {
    cout << "\n3D Transformations on cube (side 100, corner at origin)\n";
    cout << "1. Translation  2. Scaling  3. Rotation about X\n";
    cout << "4. Rotation about Y  5. Rotation about Z\n";
    cout << "Enter choice: ";
    int ch; cin >> ch;
    if (ch == 1) {
        float tx, ty, tz;
        cout << "Enter tx ty tz: "; cin >> tx >> ty >> tz;
        mTranslate(tx, ty, tz);
    } else if (ch == 2) {
        float sx, sy, sz;
        cout << "Enter sx sy sz: "; cin >> sx >> sy >> sz;
```

```
            mScale(sx, sy, sz);
        } else {
            float ang;

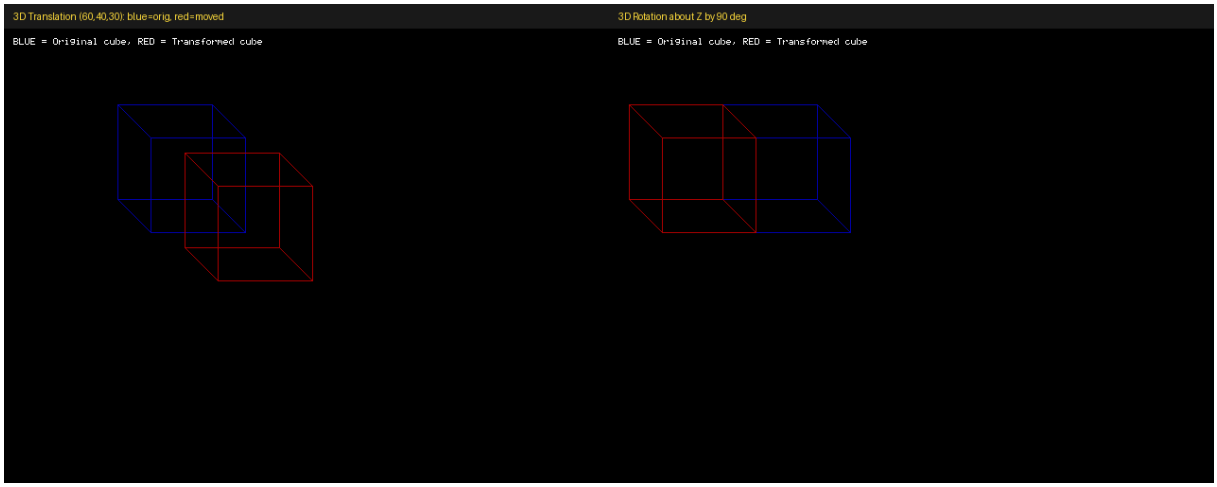
            cout << "Enter angle (degrees): "; cin >> ang;
            if (ch == 3) mRotX(ang); else if (ch == 4) mRotY(ang); else mRotZ(ang);
        }
        printMat();

        Vec3 tv[8];
        cout << "\nOriginal -> Transformed vertices:\n";
        for (int i = 0; i < 8; i++) {
            tv[i] = mApply(cube[i]);
            cout << "(" << cube[i].x << "," << cube[i].y << "," << cube[i].z << ") -> ("
                 << tv[i].x << "," << tv[i].y << "," << tv[i].z << ")\n";
        }

        int gd = DETECT, gm;
        initgraph(&gd, &gm, NULL);
        setcolor(WHITE);
        outtextxy(10, 10, "BLUE = Original cube, RED = Transformed cube");
        drawCube(cube, BLUE, 120, 80);
        delay(1200);
        drawCube(tv, RED, 120, 80);
        getch();
        closegraph();
    }

    int main() {
        cout << "*** 2D and 3D TRANSFORMATIONS ***\n";
        cout << "1. 2D (triangle)\n2. 3D (cube)\nEnter choice: ";
        int mode; cin >> mode;
        if (mode == 1) transformations2D();
        else transformations3D();
        return 0;
    }
```

Corrected Output



3-D cube: translation (left) and rotation about Z (right). Blue = original, red = transformed.

Practical No. 8 - 2-D Transformations with Composite Matrices

Original Code

```
#include <graphics.h>
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

struct Point {
    int x, y;
};

void drawPolygon(const vector<Point> &poly, int color) {
    setcolor(color);
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        int x1 = poly[i].x, y1 = poly[i].y;
        int x2 = poly[(i+1)%n].x, y2 = poly[(i+1)%n].y;
        line(x1, y1, x2, y2);
    }
}

vector<Point> scalePolygon(const vector<Point> &poly, float sx, float sy) {
    vector<Point> result;
    for (auto &p : poly) {
        Point np;
        np.x = round(p.x * sx);
        np.y = round(p.y * sy);
        result.push_back(np);
    }
    return result;
}

vector<Point> rotatePolygon(const vector<Point> &poly, float angleDeg, int xr, int yr) {
    vector<Point> result;
    float theta = angleDeg * M_PI / 180.0;
    for (auto &p : poly) {
        Point np;
        np.x = round((p.x - xr) * cos(theta) - (p.y - yr) * sin(theta) + xr);
        np.y = round((p.x - xr) * sin(theta) + (p.y - yr) * cos(theta) + yr);
        result.push_back(np);
    }
    return result;
}

vector<Point> reflectPolygon(const vector<Point> &poly, int choice) {
    vector<Point> result;
    for (auto &p : poly) {
        Point np;
        switch(choice) {
        case 1: np.x = p.x; np.y = -p.y; break; // X-axis
        case 2: np.x = -p.x; np.y = p.y; break; // Y-axis
        case 3: np.x = -p.x; np.y = -p.y; break; // Origin
        case 4: np.x = p.y; np.y = p.x; break; // Line y = x
        }
        np.x += 250;

        np.y += 250;
        result.push_back(np);
    }
    return result;
}

int main() {
    int n;
    cout << "Enter number of polygon vertices: ";
    cin >> n;
    vector<Point> poly(n);
    cout << "Enter coordinates as x y pairs:\n";
    for (int i = 0; i < n; i++) cin >> poly[i].x >> poly[i].y;

    cout << "\nChoose Transformation:\n";
    cout << "1. Scaling\n";
    cout << "2. Rotation about arbitrary point\n";
    cout << "3. Reflection\n";
    int choice; cin >> choice;

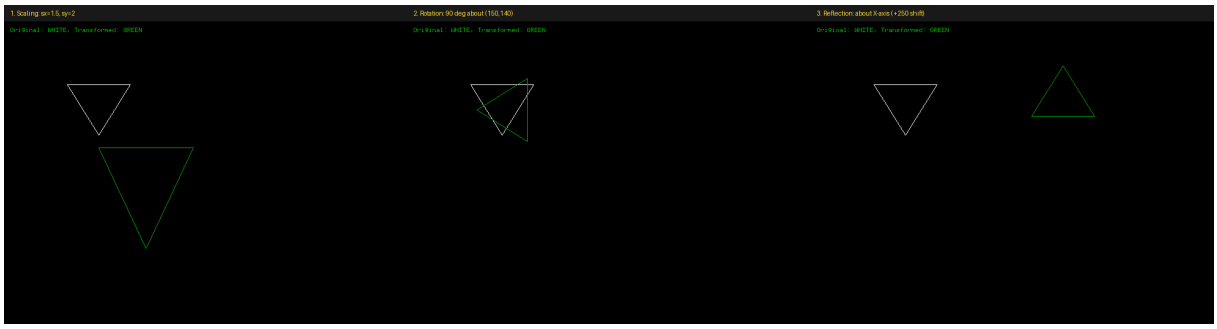
    vector<Point> transformed;
```

```
    if (choice == 1) {
        float sx, sy;
        cout << "Enter scaling factors (sx sy): ";
        cin >> sx >> sy;
        transformed = scalePolygon(poly, sx, sy);
    }
    else if (choice == 2) {
        float angle;
        int xr, yr;
        cout << "Enter rotation angle (degrees): ";
        cin >> angle;
        cout << "Enter pivot point (xr yr): ";
        cin >> xr >> yr;
        transformed = rotatePolygon(poly, angle, xr, yr);
    }
    else if (choice == 3) {
        cout << "Reflection options:\n";
        cout << "1. About X-axis\n2. About Y-axis\n3. About Origin\n4. About line y = x\n";
        int rchoice; cin >> rchoice;
        transformed = reflectPolygon(poly, rchoice);
    }

    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);
    drawPolygon(poly, WHITE);

    delay(1000);
    drawPolygon(transformed, GREEN);
    outtextxy(10, 10, "Original: WHITE, Transformed: GREEN");
    getch();
    closegraph();
    return 0;
}
```

Original Output



Scaling, rotation about a point, reflection (white = original, green = transformed)

Corrected Code (matches theory: true 3x3 composite matrices, 5 reflections, axes at centre)

```
// Practical 8 (theory version): 2-D transformations with 3x3 homogeneous
// matrices, exactly as per theory:
//   Scaling about fixed point (xf,yf): M = T(xf,yf) * S(sx,sy) * T(-xf,-yf)
//   Rotation about pivot (px,py):      M = T(px,py) * R(theta) * T(-px,-py)
//   Reflections: X-axis, Y-axis, origin, line y=x, line y=-x
// The composite matrix and new coordinates are printed. X/Y axes are drawn
// with the origin at the screen centre; original object blue, transformed red.
#include <graphics.h>
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

struct Mat { float m[3][3]; };

Mat matMul(const Mat &A, const Mat &B) {
    Mat C;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++) {
            C.m[i][j] = 0;
            for (int k = 0; k < 3; k++) C.m[i][j] += A.m[i][k] * B.m[k][j];
        }
    return C;
}
Mat matTranslate(float tx, float ty) {
    Mat T = {{{1,0,tx},{0,1,ty},{0,0,1}}};
    return T;
}
Mat matScale(float sx, float sy) {
    Mat S = {{{sx,0,0},{0,sy,0},{0,0,1}}};
    return S;
}
Mat matRotate(float deg) {
    float r = deg * 3.14159 / 180.0, c = cos(r), s = sin(r);
    Mat R = {{{c,-s,0},{s,c,0},{0,0,1}}};
    return R;
}
Mat matReflectX()    { Mat M = {{{1,0,0},{0,-1,0},{0,0,1}}}; return M; }
Mat matReflectY()    { Mat M = {{{-1,0,0},{0,1,0},{0,0,1}}}; return M; }
Mat matReflectOrig() { Mat M = {{{-1,0,0},{0,-1,0},{0,0,1}}}; return M; }
Mat matReflectYX()   { Mat M = {{{0,1,0},{1,0,0},{0,0,1}}}; return M; }  // y = x
Mat matReflectYnegX(){ Mat M = {{{0,-1,0},{-1,0,0},{0,0,1}}}; return M; } // y = -x

struct Pt { int x, y; };
Pt applyMat(const Mat &M, Pt p) {
    Pt q;
    q.x = (int)round(M.m[0][0]*p.x + M.m[0][1]*p.y + M.m[0][2]);
    q.y = (int)round(M.m[1][0]*p.x + M.m[1][1]*p.y + M.m[1][2]);
    return q;
}
void printMat(const Mat &M) {
    cout << "Composite matrix:\n";
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) cout << M.m[i][j] << "\t";

        cout << "\n";
    }
}

// math coords -> screen coords (origin at centre)
int sx_(int x) { return 320 + x; }
int sy_(int y) { return 240 - y; }

void drawPoly(const vector<Pt> &poly, int color) {
    setcolor(color);
    for (size_t i = 0; i < poly.size(); i++)
        line(sx_(poly[i].x), sy_(poly[i].y),
             sx_(poly[(i+1)%poly.size()].x), sy_(poly[(i+1)%poly.size()].y));
}

int main() {
    int n;
    cout << "Enter number of polygon vertices: ";
    cin >> n;
    vector<Pt> poly(n);
    cout << "Enter vertices in math coordinates (x y), origin at centre:\n";
    for (int i = 0; i < n; i++) cin >> poly[i].x >> poly[i].y;
```

```
    cout << "\nChoose Transformation:\n1. Scaling about fixed point\n"
         << "2. Rotation about arbitrary point\n3. Reflection\n";
    int choice; cin >> choice;

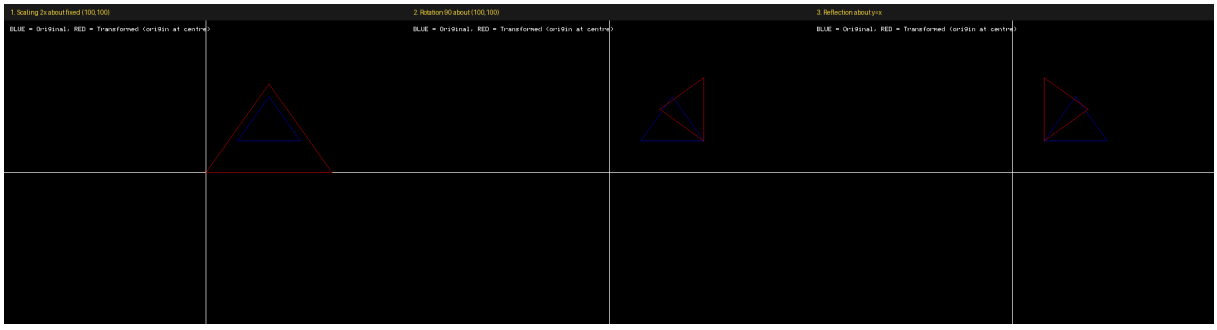
    Mat M;
    if (choice == 1) {
        float sxx, syy; int xf, yf;
        cout << "Enter sx sy and fixed point xf yf: ";
        cin >> sxx >> syy >> xf >> yf;
        M = matMul(matTranslate(xf, yf),
                   matMul(matScale(sxx, syy), matTranslate(-xf, -yf)));
    } else if (choice == 2) {
        float ang; int px, py;
        cout << "Enter angle (degrees) and pivot px py: ";
        cin >> ang >> px >> py;
        M = matMul(matTranslate(px, py),
                   matMul(matRotate(ang), matTranslate(-px, -py)));
    } else {
        cout << "1. X-axis  2. Y-axis  3. Origin  4. line y=x  5. line y=-x\n";
        int r; cin >> r;
        if (r == 1) M = matReflectX();
        else if (r == 2) M = matReflectY();
        else if (r == 3) M = matReflectOrig();
        else if (r == 4) M = matReflectYX();
        else M = matReflectYnegX();
    }
    printMat(M);

    vector<Pt> tp;
    cout << "Transformed vertices:\n";
    for (auto &p : poly) {

        Pt q = applyMat(M, p);
        tp.push_back(q);
        cout << "(" << p.x << "," << p.y << ") -> (" << q.x << "," << q.y << ")\n";
    }

    int gd = DETECT, gm;
    initgraph(&gd, &gm, NULL);
    setcolor(WHITE);
    line(0, 240, 639, 240);   // X-axis
    line(320, 0, 320, 479);   // Y-axis
    outtextxy(10, 10, "BLUE = Original, RED = Transformed (origin at centre)");
    drawPoly(poly, BLUE);
    delay(1000);
    drawPoly(tp, RED);
    getch();
    closegraph();
    return 0;
}
```

Corrected Output



Composite-matrix transforms: scaling about fixed point, rotation about pivot, reflection about y=x (blue = original, red = transformed)

Ubuntu Commands - All Practicals

One-time setup: `sudo apt update && sudo apt install g++`
Keep `graphics.h` (and `conio.h` for Practical 7) in the SAME folder as the .cpp file. On Ubuntu, `graphics.h` is a compatibility header: the same functions as Turbo C, but the drawing is saved to an image file (.ppm) instead of opening a window.

```
# ---------- Practical 2 ----------
g++ practical2.cpp -o practical2 -lm -I.
./practical2


# ---------- Practical 3 ----------
g++ practical3.cpp -o practical3 -lm -I.
./practical3


# ---------- Practical 4 ----------
g++ practical4.cpp -o practical4 -lm -I.
./practical4


# ---------- Practical 5 (original) ----------
g++ point_clip.cpp -o point_clip -lm -I.               ; ./point_clip
g++ cohen_sutherland.cpp -o cohen_sutherland -lm -I.   ; ./cohen_sutherland
g++ liang_barsky.cpp -o liang_barsky -lm -I.           ; ./liang_barsky
g++ sutherland_hodgman.cpp -o sutherland_hodgman -lm -I. ; ./sutherland_hodgman


# ---------- Practical 5 (corrected: world window -100..100, grey/red) ----------
g++ p5_point.cpp -o p5_point -lm -I.           ; ./p5_point
g++ p5_cohen.cpp -o p5_cohen -lm -I.           ; ./p5_cohen
g++ p5_liang.cpp -o p5_liang -lm -I.           ; ./p5_liang
g++ p5_sutherland.cpp -o p5_sutherland -lm -I. ; ./p5_sutherland


# ---------- Practical 6 (original) ----------
g++ practical6.cpp -o practical6 -lm -I.
./practical6


# ---------- Practical 6 (corrected: mouse + keyboard, 800x400) ----------
g++ p6_interactive.cpp -o p6_interactive -lm -I.
./p6_interactive
# headless Ubuntu demo with scripted mouse/keyboard:
BGI_SIZE=800x400 BGI_SCRIPT=demoA.txt ./p6_interactive


# ---------- Practical 7 (original, 2-D only) ----------
g++ practical7.cpp -o practical7 -lm -I.
./practical7


# ---------- Practical 7 (corrected: 2-D + 3-D) ----------
g++ p7_23d.cpp -o p7_23d -lm -I.
./p7_23d


# ---------- Practical 8 (original) ----------
g++ practical8.cpp -o practical8 -lm -I.
./practical8


# ---------- Practical 8 (corrected: composite matrices) ----------
g++ p8_matrix.cpp -o p8_matrix -lm -I.
./p8_matrix
```